

ELDEN RING

Phase 1: Runtime Defense Plane

구현 완료 보고서

프로젝트 : ELDEN RING - AI 기반 능동 방어 및 시큐어 코딩 시스템
단계 : Phase 1 - Runtime Defense Plane (런타임 방어 플레인)
버전 : v2.0.0
상태 : 구현 완료 (Feature-Complete)
작성일 : 2026년 4월 12일
작성팀 : MJSEC
브랜치 : feature/phase1-integration-verified

1. 프로젝트 개요

ELDEN RING은 Kubernetes 기반 AI-to-AI 능동 방어 및 시큐어 코딩 자동화 시스템으로, 4단계 자동화 보안 파이프라인을 통해 공격 탐지부터 패치 적용까지 전 과정을 자동화합니다.

Phase 1(Runtime Defense Plane)은 파이프라인의 최전방 단계로서, 웹 애플리케이션에 대한 공격을 실시간으로 탐지하고, 이벤트를 정규화하며, CWE 식별자를 매핑하고, 구조화된 컨텍스트 패키지를 Phase 2에 전달하는 역할을 수행합니다.

4단계 파이프라인 아키텍처

Phase	이름	네임스페이스	담당자	상태
Phase 1	Runtime Defense	elden-production	이주오	완료
Phase 2	Secure Coding	elden-secure-coding	이윤태	예정
Phase 3	Recovery Assurance	elden-staging	이윤태	예정
Phase 4	Governance	elden-governance	이종윤	예정

2. Phase 1 구현 범위 및 아키텍처

Phase 1은 2중 탐지 체계(ModSecurity WAF + Falco)를 통해 공격을 탐지하고, 6단계 이벤트 처리 파이프라인을 거쳐 구조화된 컨텍스트를 생성합니다.

이벤트 처리 파이프라인 (6단계)

- 이벤트 수집 (Ingestion) - ModSecurity/Falco/수동 주입 3개 진입점
- 이벤트 정규화 (Normalization) - Adapter 패턴 기반 통합 포맷 변환
- CWE 매핑 (Mapping) - 규칙 테이블 기반 100% 결정적 매핑 (7개 공격 유형)
- 소스코드 매핑 (Source Mapping) - 엔드포인트 → 파일:함수:라인 변환
- 컨텍스트 조립 (Context Building) - Phase 2 호환 JSON 구조 생성
- 능동 방어 (Active Defense) - 3단계 에스컬레이션 대응

2중 탐지 체계

구분	탐지 도구	탐지 레벨	역할
1차 방어	ModSecurity WAF	HTTP 요청	SQLi, XSS, Path Traversal 차단
2차 방어	Falco	시스템 콜	셸 실행, 권한 상승, 비정상 연결 탐지

3. 핵심 컴포넌트 상세

3-1. Runtime Defense Controller

FastAPI 기반 중앙 이벤트 처리 허브로, ModSecurity 감사 로그와 Falco 웹훅을 수신하여 정규화, CWE 매핑, 소스 매핑, 능동 방어를 수행합니다.

언어/프레임워크	Python 3.11 / FastAPI
프로덕션 코드	471줄 (11개 모듈)
테스트 코드	509줄 (7개 테스트 모듈)
컨테이너 이미지	eldenring/runtime-defense:latest
포트	8080 (HTTP)

API 엔드포인트 (10개)

엔드포인트	메서드	용도
/api/v1/modsec-events	POST	ModSecurity 감사 로그 수신
/api/v1/falco-events	POST	Falco Sidekick 웹훅 수신
/api/v1/events/manual	POST	수동 이벤트 주입 (데모)
/api/v1/contexts/{id}	GET	특정 컨텍스트 조회
/api/v1/contexts/latest	GET	최근 N개 컨텍스트 조회
/api/v1/defense/actions	GET	방어 조치 이력 조회
/api/v1/defense/stats	GET	방어 통계 조회
/api/v1/events/stats	GET	이벤트 집계 통계
/healthz	GET	헬스 체크
/readyz	GET	레디니스 체크

3-2. Target Application (의도적 취약 앱)

Phase 1 탐지 테스트를 위한 Flask 기반 의도적 취약 애플리케이션입니다.

엔드포인트	메서드	CWE	취약점	테스트 페이로드
/api/login	POST	CWE-89	SQL Injection	admin' OR 1=1--
/api/search	GET	CWE-79	Reflected XSS	<script>alert(1)</script>
/api/file	GET	CWE-22	Path Traversal	../../../../etc/passwd

4. 능동 방어 시스템 (Active Defense)

공격 이벤트 발생 시 심각도와 빈도에 따라 3단계로 에스컬레이션되는 능동 방어 체계를 구현하였습니다.

레벨	방어 조치	발동 조건	구현 방식
Lv.1	Rate Limit (10 RPM)	모든 이벤트	요청 속도 제한
Lv.2	IP 차단	IP당 3회 이상 또는 HIGH/CRITICAL	NetworkPolicy 동적 생성
Lv.3	엔드포인트 비활성화	엔드포인트 5회 이상 또는 CRITICAL	Ingress 패치

5. Kubernetes 인프라 구성

배포 리소스

리소스	이름	네임스페이스	비고
Deployment	runtime-defense-controller	elden-production	CPU 200m/500m, MEM 256Mi/512Mi
Deployment	target-app (x2 replicas)	elden-production	CPU 100m/250m, MEM 128Mi/256Mi
Service	runtime-defense-svc	elden-production	ClusterIP, Port 8080
Service	target-app-svc	elden-production	ClusterIP, Port 5000
Ingress	target-app-ingress	elden-production	ModSecurity WAF 적용
ConfigMap	route-map	elden-production	엔드포인트-소스코드 매핑

보안 구성

- Istio 서비스 메시: mTLS STRICT 모드, AuthorizationPolicy 적용
- RBAC: runtime-defense-sa ServiceAccount, NetworkPolicy/Ingress CRUD 권한
- NetworkPolicy: 네임스페이스 간 트래픽 격리
- ResourceQuota: 네임스페이스별 리소스 사용량 제한

모니터링 스택

- Prometheus: 메트릭 수집 (elden-monitoring 네임스페이스)
- Grafana: 대시보드 시각화
- Loki: 로그 집계
- Redis: 메시지 브로커 (Pub/Sub + List 큐)

6. CI/CD 파이프라인

워크플로우	트리거	주요 작업
dev-ci.yaml	dev 브랜치 Push	Docker 빌드, Trivy 스캔, K8s 배포
pr-validation.yaml	Pull Request	YAML lint, kubeval, 유닛 테스트
main-promote.yaml	main 머지	dev-latest → production 태그 승격

경로 기반 변경 감지(dorny/paths-filter)를 적용하여 변경된 서비스만 빌드/배포합니다. Trivy를 통한 컨테이너 이미지 보안 스캔(HIGH, CRITICAL 레벨)이 자동 수행됩니다.

7. 테스트 현황

테스트 파일	라인 수	테스트 대상
test_adapters.py	166	ModSecurity/Falco 어댑터 파싱
test_defense.py	95	3단계 방어 에스컬레이션 로직
test_source_mapper.py	68	엔드포인트 → 소스코드 매핑

test_context_builder.py	61	컨텍스트 패키지 조립
test_normalizer.py	39	어댑터 라우팅
test_cwe_mapping.py	39	CWE 규칙 테이블
test_redis_publisher.py	41	Redis 이중 전달 및 장애 복구

총 테스트 코드 509줄 (7개 모듈)

테스트 커버리지 핵심 파이프라인 전 모듈 단위 테스트 완료

8. 최근 작업 이력 (Git 커밋)

커밋 해시	유형	내용
6b410c3	fix	Kind 클러스터 배포 이슈 해결 (base infra + Falco 설정)
853b54b	feat	ModSecurity WAF + OWASP CRS 인그레스 컨트롤러 활성화
ee02dbe	feat	Runtime Defense 컨트롤러, target-app, K8s 매니페스트 구현
32f4393	refactor	레거시 elden-ring-infra 하위 디렉토리 제거

9. Phase 2 연동 (컨텍스트 패키지 스키마)

Phase 1은 아래의 구조화된 JSON 컨텍스트 패키지를 Redis를 통해 Phase 2에 전달합니다. Pub/Sub(실시간)과 List 큐(영속성) 이중 채널을 사용합니다.

Pub/Sub 채널 elden:phase2:context

List 큐 elden:phase2:context:queue

Redis 주소 redis-master.elden-monitoring:6379

컨텍스트 패키지 주요 필드

필드	타입	설명
context_id	string	고유 컨텍스트 ID (ctx-evt-{source}-{uuid})
event_id	string	원본 이벤트 ID
timestamp	ISO 8601	이벤트 발생 시각
attack_info.cwe_id	string	CWE 식별자 (예: CWE-89)
attack_info.category	string	공격 분류 (SQL Injection 등)
attack_info.owasp_category	string	OWASP 분류 (A03:2021 등)
attack_info.payload_sample	string	공격 페이로드 샘플
target.source_mapping	object	file, function, line_start, line_end
metadata.severity	string	심각도 (CRITICAL/HIGH/MEDIUM/LOW)
metadata.requires_patch	boolean	패치 생성 필요 여부

10. 코드 메트릭 요약

항목	수치
프로덕션 코드 (Runtime Defense)	471줄
프로덕션 코드 (Target App)	154줄
테스트 코드	509줄
Kubernetes YAML 매니페스트	1,770줄
CI/CD 워크플로우	438줄 (3개 파일)
문서	1,856줄 (2개 파일)
API 엔드포인트	10개
CWE 매핑 규칙	7개
탐지 공격 유형	3개 (SQLi, XSS, Path Traversal)
능동 방어 레벨	3단계
K8s 네임스페이스	7개 (4 Phase + 3 System)

11. 미완료 항목 및 후속 작업

항목	상태	우선순위	비고
Fluent Bit 사이드카 배포	미완료	중간	ModSecurity 로그 자동 수집 (수동 수집 가능)
Grafana 대시보드 생성	미완료	낮음	모니터링 인프라 준비 완료
E2E 통합 테스트	미완료	높음	ModSecurity → Phase1 → Redis 전체 흐름
Phase 2 연동 테스트	대기	높음	Phase 2 구현 후 진행

12. 결론

Phase 1 Runtime Defense Plane은 계획된 모든 핵심 기능의 구현을 완료하였습니다. 2중 탐지 체계(ModSecurity WAF + Falco), 6단계 이벤트 처리 파이프라인, 3단계 능동 방어 에스컬레이션, Redis 기반 Phase 2 연동이 모두 구현되었으며, 7개 테스트 모듈을 통한 단위 테스트가 완료되었습니다.

현재 Phase 2(Secure Coding) 핸드오프 준비가 완료된 상태이며, Phase 2 통합 가이드 문서와 컨텍스트 패키지 스키마가 제공되어 있습니다. 후속 작업으로 Fluent Bit 사이드카 배포와 E2E 통합 테스트가 필요하며, 이는 Phase 2 구현과 병행하여 진행할 수 있습니다.